ShortsBlocker — Technical Specification

Package: `com.shortsblocker.app` · Platform: Android (minSdk 26 / targetSdk 34) · Version: 0.1.0

This document specifies the functional requirements of the ShortsBlocker app and the test/use cases that verify them. It reflects the current implementation in `app/src/main/java/com/shortsblocker/app/`.

---

1. Overview

ShortsBlocker is an on-device Android app that detects and blocks short-form video content (YouTube Shorts, Instagram Reels, TikTok, Facebook Reels, Snapchat) using an `AccessibilityService`, while preserving normal use of the same apps (long-form YouTube, Instagram DMs/feed, Facebook Messenger/feed). All detection runs locally against a bundled rule file; no screen content or personal data ever leaves the device.

1.1 System Components

Layer	Component	Responsibility
Rule engine	`rules/RuleEvaluator`, `rules/BlockRule`, `rules/BlockDecision`	Pure, framework-independent evaluation of a node tree against a `RuleSet`
Rule data	`assets/block_rules.json`, `rules/RuleRepository`, `rules/RuleSetParser`	Bundled, versioned rule definitions (no remote fetch)
Detection	`service/ShortsBlockAccessibilityService`, `rules/AccessibilityNodeAdapter`	Listens for `AccessibilityEvent`s, adapts the live node tree, runs the evaluator
Enforcement	`service/BlockActionExecutor`, `overlay/BlockOverlayController`	Performs BACK/HOME actions and shows a transient overlay when content is blocked
Persistence (service)	`service/BlockGuardForegroundService`, `service/BootCompletedReceiver`	Keeps the accessibility service alive; restores it after device reboot
Preferences	`prefs/BlockingPreferences`, `prefs/SupportedApps`	Master on/off state and per-app enable/disable state
Security	`security/PinManager`, `security/HashUtils`	PIN + recovery-question storage (salted hash, `EncryptedSharedPreferences`)
Logging	`logging/BlockEventLogger`	On-device-only ring buffer of block/unmatched events
UI	`ui/MainActivity`, `ui/OnboardingScreen`, `ui/PinScreens`, `ui/HomeScreen`, `ui/SettingsScreen`	Jetpack Compose screens, navigated via `AppNavGraph`

---

2. Functional Requirements

Each requirement has a unique ID, referenced by the test/use cases in Section 3.

2.1 Content Detection & Blocking

ID	Requirement	Source
FR-01	The app SHALL block the YouTube Shorts player, Shorts feed, and Shorts tab, while allowing the long-form watch player, comments, subscriptions, and search results.	`block_rules.json` → `com.google.android.youtube`
FR-02	The app SHALL block the Instagram Reels viewer and Reels tab, while allowing Direct Messages (`direct_*`, `inbox`, `thread_*`) and the normal feed.	`block_rules.json` → `com.instagram.android`
FR-03	The app SHALL block TikTok (`com.zhiliaoapp.musically`) and the TikTok Lite/regional variant (`com.ss.android.ugc.trill`) in full — the app is blocked entirely, with no partial allowance.	`block_rules.json` (`mode: FULL`)
FR-04	The app SHALL block the Facebook Reels player and Reels tab, while allowing Messenger and the News Feed.	`block_rules.json` → `com.facebook.katana`
FR-05	The app SHALL block Snapchat (`com.snapchat.android`) in full.	`block_rules.json` (`mode: FULL`)
FR-06	The app SHALL detect short-form URLs (`youtube.com/shorts`, `instagram.com/reel(s)/`, `tiktok.com`) typed or loaded into the URL bar of supported browsers (Chrome, Samsung Internet, Firefox, Edge) and block them.	`block_rules.json` → `browsers` / `browserBlockUrlPatterns`
FR-07	X/Twitter SHALL be excluded from blocking (explicit product decision).	`docs/DECISIONS.md`
FR-08	Whitelist rules SHALL always take priority over block rules for the same package.	`RuleEvaluator.evaluatePackage`
FR-09	A package/screen that matches neither a whitelist nor a block rule SHALL be treated as Unmatched (pass-through) and logged, never silently blocked or silently allowed without a record.	`BlockDecision.Unmatched`
FR-10	Rule definitions SHALL ship bundled inside the APK (`assets/block_rules.json`) only. The app SHALL NOT fetch rules from a remote server.	`RuleRepository`, `docs/DECISIONS.md`

2.2 Enforcement

ID	Requirement	Source
FR-11	When a block decision is made, the app SHALL perform the global BACK action and show a brief on-screen overlay notification.	`BlockActionExecutor.onBlockDetected`
FR-12	If the blocked screen is still active 200 ms after BACK, the app SHALL perform the global HOME action as an escalation.	`BlockActionExecutor` (`RECHECK_DELAY_MS = 200`)
FR-13	Repeated triggers for the same package + rule within 1000 ms SHALL be debounced (ignored) to prevent action loops.	`BlockActionExecutor` (`DEBOUNCE_MS = 1000`)
FR-14	The block overlay SHALL never render captured screen content — text only.	`BlockOverlayController`
FR-15	Node-tree traversal for a single evaluation SHALL be bounded to at most 500 nodes and 60 levels of depth, to bound per-event processing cost.	`RuleEvaluator(maxNodesVisited = 500, maxDepth = 60)`

2.3 Master & Per-App Control

ID	Requirement	Source
FR-16	The Home screen SHALL show a master on/off switch, visually reduced in size relative to standard controls.	`HomeScreen` (`Modifier.scale(0.75f)`)
FR-17	Turning the master switch ON SHALL take effect immediately, with no confirmation or PIN required.	`HomeScreen`
FR-18	Turning the master switch OFF SHALL require, in order: (a) a confirmation dialog, then (b) successful PIN entry. The switch SHALL NOT actually disable blocking from the confirmation dialog alone.	`HomeScreen`, `MainActivity` (`Routes.DISABLE_PIN_ENTRY`)
FR-19	When the master switch is OFF, the accessibility service SHALL skip evaluation entirely for all packages.	`ShortsBlockAccessibilityService.onAccessibilityEvent`
FR-20	Settings SHALL provide an independent enable/disable toggle for each supported app (YouTube, Instagram, TikTok, Facebook, Snapchat).	`SettingsScreen`, `SupportedApps`
FR-21	When a given app's toggle is OFF, the accessibility service SHALL skip evaluation for that package specifically, independent of the master switch or other apps' toggles.	`ShortsBlockAccessibilityService.onAccessibilityEvent`, `BlockingPreferences.isPackageEnabled`
FR-22	Per-app toggles marked with reduced-accuracy notes (e.g., Facebook: "detection accuracy may be lower") SHALL surface that caveat in the Settings UI.	`SupportedApps.note`, `SettingsScreen`

2.4 PIN & Recovery

ID	Requirement	Source
FR-23	On first launch (no PIN set), the app SHALL route the user through onboarding, then require setting a PIN of at least 4 numeric digits.	`MainActivity` (`Routes.ONBOARDING` → `PIN_SETUP`), `PinSetupScreen`
FR-24	PIN setup SHALL require a non-blank recovery question and answer; the PIN cannot be created without them.	`PinSetupScreen`
FR-25	The PIN and recovery answer SHALL be stored only as salted hashes (`HashUtils`, PBKDF2WithHmacSHA256, 120,000 iterations) inside `EncryptedSharedPreferences` (AES-256-GCM / AES-256-SIV). Plaintext SHALL never be persisted or transmitted off-device.	`PinManager`, `HashUtils`
FR-26	Access to the Settings screen SHALL require successful PIN entry.	`MainActivity` (`Routes.PIN_ENTRY` → `SETTINGS`)
FR-27	If the user forgets their PIN, the app SHALL allow recovery via the stored security question/answer (case-insensitive, trimmed comparison) — no email or SMS channel is used.	`PinRecoveryScreen`, `PinManager.verifyRecoveryAnswer`
FR-28	Successful recovery SHALL route the user to set a brand-new PIN (and new recovery Q&A).	`MainActivity` (`Routes.PIN_RECOVERY` → `PIN_SETUP`)
FR-29	An incorrect PIN or recovery answer SHALL show an inline error and SHALL NOT navigate forward.	`PinScreens.kt`

2.5 Status, Logging & Permissions

ID	Requirement	Source
FR-30	The Home screen SHALL display whether the Accessibility Service is currently enabled in system settings (Active/Inactive), re-checked on every `ON_RESUME`.	`HomeScreen`, `AccessibilityStatus.isServiceEnabled`
FR-31	The Home screen SHALL display the most recent on-device block/unmatched events (timestamp, package, result), most recent first, re-read on every `ON_RESUME`.	`HomeScreen`, `BlockEventLogger.recentEvents`
FR-32	The event log SHALL store only package name, matched rule ID (or the literal string `"UNMATCHED"`), and timestamp — never screen text, view content, or other personal data.	`BlockEventLogger`
FR-33	The event log SHALL retain at most the 50 most recent events (ring buffer), oldest entries dropped first.	`BlockEventLogger` (`MAX_EVENTS = 50`)
FR-34	Settings SHALL provide direct deep-links to the system Accessibility settings screen and the "draw over other apps" (overlay) permission screen.	`SettingsScreen`
FR-35	A foreground service (`BlockGuardForegroundService`, `foregroundServiceType="specialUse"`) SHALL keep the accessibility service's process alive.	`AndroidManifest.xml`, `BlockGuardForegroundService`
FR-36	On device boot (`BOOT_COMPLETED`), the app SHALL re-establish its blocking service without requiring the user to reopen the app.	`BootCompletedReceiver`

---

3. Use Cases / Test Cases

Each case lists preconditions, steps, and expected result, and is traced to the functional requirement(s) it verifies. Cases marked [Automated] exist as JUnit tests in `app/src/test/java/com/shortsblocker/app/rules/RuleEvaluatorTest.kt` and run via `./gradlew test`. Cases marked [Manual] must be verified on-device.

3.1 Rule Engine

Case ID	FR	Type	Precondition	Steps	Expected Result
TC-01	FR-01	Automated	YouTube rule loaded	Node tree contains `reel_player_page`	Decision = `Block("yt_shorts_player")`
TC-02	FR-01, FR-08	Automated	YouTube rule loaded	Tree contains both `watch_player` (whitelist) and a `Shorts`-labeled nav item	Decision = `Allow(whitelistMatched = true)` — whitelist wins
TC-03	FR-01	Automated	YouTube rule loaded	Tree contains a `bottom_bar_item` labeled `"Home"`	Decision = `Unmatched` (Shorts-tab rule requires matching text, so an unrelated tab is not blocked)
TC-04	FR-01	Automated	YouTube rule loaded	Tree contains a `bottom_bar_item` labeled `"Shorts"`	Decision = `Block`
TC-05	FR-02, FR-08	Automated	Instagram rule loaded	Tree contains both a `direct_thread_view` node and a `clips_viewer_view_pager` node	Decision = `Allow(whitelistMatched = true)` — DM presence always wins even alongside a Reels node
TC-06	FR-02	Automated	Instagram rule loaded	Tree contains only `clips_viewer_view_pager`	Decision = `Block("ig_reels_viewer")`
TC-07	FR-02, FR-09	Automated	Instagram rule loaded	Tree contains only `feed_recycler` (no rule matches)	Decision = `Unmatched`
TC-08	FR-03	Automated	TikTok rule loaded (`FULL`)	Tree contains an unrelated node (`direct_thread_view`)	Decision = `Block` regardless of tree contents
TC-09	FR-04	Automated	Facebook rule loaded	Tree contains `reels_player_view`	Decision = `Block("fb_reels_viewer")`
TC-10	FR-04	Automated	Facebook rule loaded	Tree contains `newsfeed_recycler_view`	Decision = `Allow(whitelistMatched = true)`
TC-11	FR-05	Automated	Snapchat rule loaded (`FULL`)	Any/empty tree	Decision = `Block`, regardless of tree contents
TC-12	FR-09	Automated	Rule set loaded	Package not present in `RuleSet.packages`	Decision = `Unmatched`
TC-13	FR-06	Automated	Chrome browser rule loaded	URL bar node text = `"youtube.com/shorts/abc123"`	Decision = `Block`

Case ID	FR	Type	Precondition	Steps	Expected Result
TC-14	FR-06	Automated	Chrome browser rule loaded	URL bar node text = `"youtube.com/watch?v=abc123"`	Decision = `Unmatched`
TC-15	FR-15	Automated	Evaluator bounded to `maxNodesVisited = 2`	Tree has 3+ nodes, target match is the 3rd node visited	Decision = `Unmatched` (bound reached before match); `visitedNodes.size <= 2`
TC-16	(integration correctness)	Automated	Instagram Reels node present	Run evaluation, then call `.recycle()` on every node in `result.visitedNodes`	Both root and child report `recycled == true` — no node is leaked/skipped by early-return branches

3.2 Enforcement Behavior

Case ID	FR	Type	Precondition	Steps	Expected Result
TC-17	FR-11, FR-12	Manual	Accessibility service active, master + app toggle ON	Open YouTube → tap Shorts tab	Screen is backed out (BACK) within ~200 ms; overlay message briefly appears; if still on Shorts, device returns HOME
TC-18	FR-13	Manual	Same as TC-17	Rapidly re-trigger the same Shorts screen within 1 second	Only the first trigger fires BACK/HOME/overlay; subsequent triggers within the debounce window are ignored
TC-19	FR-14	Manual	Block triggered	Inspect the overlay view	Overlay shows only static text (`overlay_message`); no screenshot/screen-mirroring content is rendered

3.3 Master & Per-App Toggles

Case ID	FR	Type	Precondition	Steps	Expected Result
TC-20	FR-16	Manual	On Home screen	Visually inspect the master switch	Switch renders at 0.75× scale, smaller than default Material3 size
TC-21	FR-17	Manual	Master switch OFF	Tap switch to turn ON	Blocking resumes immediately; no dialog or PIN prompt shown
TC-22	FR-18	Manual	Master switch ON	Tap switch to turn OFF	Confirmation dialog appears; dismissing/cancelling leaves blocking ON
TC-23	FR-18	Manual	Confirmation dialog shown (from TC-22)	Tap "Yes"	Navigates to PIN entry; blocking remains ON until PIN is verified
TC-24	FR-18	Manual	On PIN entry from disable flow	Enter correct PIN	`masterEnabled` becomes `false`; navigates back to Home; switch now shows OFF
TC-25	FR-18	Manual	On PIN entry from disable flow	Enter incorrect PIN	Inline error shown; `masterEnabled` remains `true`; blocking stays ON
TC-26	FR-19	Manual	Master OFF (per TC-24)	Open TikTok	No BACK/HOME/overlay triggered; app usable normally
TC-27	FR-20, FR-21	Manual	Settings open, master ON	Toggle "Facebook" OFF, leave others ON	Facebook Reels no longer blocked; YouTube/Instagram/TikTok/Snapchat still blocked
TC-28	FR-22	Manual	Settings open	Inspect Facebook and Snapchat rows	Facebook row shows "Detection accuracy may be lower"; Snapchat row shows "Blocks the entire app"

3.4 PIN & Recovery Flows

Case ID	FR	Type	Precondition	Steps	Expected Result
TC-29	FR-23	Manual	Fresh install, no PIN set	Launch app	Routes to Onboarding, then PIN Setup; Home is unreachable directly
TC-30	FR-23, FR-24	Manual	On PIN Setup screen	Enter a 3-digit PIN, leave question/answer blank, submit	Error: "PIN must be at least 4 digits"; form not submitted
TC-31	FR-24	Manual	On PIN Setup screen	Enter valid 4+ digit PIN, leave recovery question blank, submit	Error: "Enter a recovery question and answer"; form not submitted
TC-32	FR-23, FR-24	Manual	On PIN Setup screen	Enter valid PIN + question + answer, submit	PIN and recovery hash saved; navigates to Home; app restart routes directly to Home (`isPinSet() == true`)
TC-33	FR-26	Manual	PIN set, on Home	Tap "Settings"	Routes to PIN Entry before Settings is shown
TC-34	FR-26, FR-29	Manual	On PIN Entry (Settings path)	Enter correct PIN	Navigates to Settings
TC-35	FR-29	Manual	On PIN Entry	Enter incorrect PIN	Inline error shown ("PIN doesn't match"); stays on PIN Entry
TC-36	FR-27	Manual	On PIN Entry	Tap "Forgot PIN"	Navigates to Recovery screen, displaying the stored recovery question
TC-37	FR-27, FR-29	Manual	On Recovery screen	Enter wrong answer	Inline error shown; PIN unchanged
TC-38	FR-27, FR-28	Manual	On Recovery screen	Enter correct answer (case/white space-insensitive)	Navigates to PIN Setup to create a new PIN + new recovery Q&A; old PIN no longer valid
TC-39	FR-25	Manual (code inspection)	—	Inspect `pin_store EncryptedShared Preferences` contents on device (e.g., via `adb shell`)	Only encrypted blobs are visible; no plaintext PIN or answer recoverable without the Keystore-backed master key

3.5 Status, Logging, Permissions & Persistence

Case ID	FR	Type	Precondition	Steps	Expected Result
TC-40	FR-30	Manual	Accessibility permission not yet granted	Open Home	Status text shows "Inactive" equivalent
TC-41	FR-30	Manual	Grant Accessibility permission, return to app	Resume Home (`ON_RESUME`)	Status updates to "Active" without app restart
TC-42	FR-31, FR-32, FR-33	Manual	Several blocks/unmatched screens triggered	Open Home	Event list shows timestamp + package + result (`Block:<ruleId>` or `UNMATCHED`) for up to the last 50 events, newest first; no screen text present
TC-43	FR-34	Manual	On Settings	Tap "Open Accessibility Settings" / "Open Overlay Settings"	System settings screens open directly (`ACTION_ACCESSIBILITY_SETTINGS`, `ACTION_MANAGE_OVERLAY_PERMISSION`)
TC-44	FR-35, FR-36	Manual	App set up, service enabled	Reboot device, do not open the app manually	After boot completes, Accessibility status shows "Active" and blocking works without manually reopening the app
TC-45	FR-10	Manual (code inspection)	—	Inspect network calls during app use (e.g., via a proxy)	No outbound requests related to rule fetching; `block_rules.json` is read only from local `assets/`

---

4. Non-Functional Requirements

ID	Requirement
NFR-01	Rule evaluation for a single accessibility event SHALL complete without noticeably degrading UI responsiveness in the foreground app, bounded by FR-15 (≤500 nodes, ≤60 depth).
NFR-02	No screen content, personal data, or credentials SHALL be transmitted off the device at any point (detection, logging, PIN storage).
NFR-03	PIN and recovery-answer storage SHALL use platform-backed encryption (Android Keystore via `MasterKey`/`EncryptedSharedPreferences`), not plaintext `SharedPreferences`.
NFR-04	The rule set SHALL be updatable only via an app update (new APK build), never via a silent remote push.

5. Traceability Summary

- FR-01 – FR-10 (detection/blocking rules) → TC-01 – TC-16
- FR-11 – FR-15 (enforcement) → TC-17 – TC-19
- FR-16 – FR-22 (master/per-app toggles) → TC-20 – TC-28
- FR-23 – FR-29 (PIN/recovery) → TC-29 – TC-39
- FR-30 – FR-36 (status/logging/permissions/persistence) → TC-40 – TC-45

Automated cases (TC-01 – TC-16) run via:

```
./gradlew test
```

All other cases require a physical/emulated device with the Accessibility Service and overlay permission granted.